

CENTRO UNIVERSITÁRIO - ECOSSISTEMA DE ENSINO A DISTÂNCIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE DADOS

ESTUDANTE GRADUANDO

PLANO DE IMPLANTAÇÃO E GOVERNANÇA DO PORTFOLIOHUB

Integração Estratégica de Ecossistemas Web com GitHub, Google Workspace e
Inteligência Artificial Gemini sob Diretrizes de DevSecOps

DF - SAMAMBAIA

2026

ESTUDANTE GRADUANDO

PLANO DE IMPLANTAÇÃO E GOVERNANÇA DO PORTFOLIOHUB

Integração Estratégica de Ecossistemas Web com GitHub, Google Workspace e
Inteligência Artificial Gemini sob Diretrizes de DevSecOps

Trabalho de Planejamento e Engenharia de Software apresentado como requisito regulamentar para a Entrega Final do Desafio de Implantação do PortfolioHUB. Desenvolvido em conformidade com as exigências acadêmicas de integração tecnológica de ferramentas de versionamento, produtividade corporativa e segurança cibernética. Orientador e Co-piloto Estratégico: Inteligência Artificial Google Gemini.

DF - SAMAMBAIA

2026

SUMÁRIO

1 SEÇÃO 1: PLANEJAMENTO DA IMPLANTAÇÃO (PORTFOLIOHUB + IA GEMINI)	
.....	4
1.1 Definição do Escopo e Objetivos Gerais	4
1.2 Cronograma de Execução Crítica (Sprint de 48 Horas)	4
1.3 Configuração e Engenharia de Prompt para o Google Gemini	5
2 SEÇÃO 2: CONFIGURAÇÃO INICIAL E INTEGRAÇÃO COM GITHUB	
.....	6
2.1 Provisionamento do Ambiente Dedicado e Governança Inicial	6
2.2 Estrutura Arquitetural de Pastas do Workspace	6
2.3 Mecanismo de Integração Dinâmica via API REST do GitHub	6
2.4 Pipeline de Implantação e Deploy Contínuo (CI/CD)	7
3 SEÇÃO 3: GESTÃO DE USUÁRIOS E SEGURANÇA (DEVSECOPS)	
.....	8
3.1 Controle de Acesso Baseado em Regras (RBAC) e Fatores de Autenticação	8
3.2 Políticas Robustas de Proteção de Código e Ramificações	8
3.3 Varredura Automatizada de Vulnerabilidades e Vazamento de Segredos	8
3.4 Segurança de Frontend e Mitigação de Vetores de Ataque Web	9
4 SEÇÃO 4: COMPARTILHAMENTO E CONTROLE DE ACESSO COM GITHUB	
.....	10
4.1 Fluxo de Trabalho de Engenharia de Código (Git Flow Simplificado)	10
4.2 Padronização de Histórico via Commits Semânticos	10
4.3 Artefatos de Governança Comunitária e Normas de Colaboração	11
5 SEÇÃO 5: FINALIZAÇÃO DA INTEGRAÇÃO E TESTES	12
5.1 Matriz de Testes Integrados e Homologação de Funcionalidades	12
5.2 Validação de Segurança Pré-Lançamento (Audit Go-Live)	12
5.3 Transição de Ambientes e Acessibilidade Pública em Produção	13
6 SEÇÃO 6: REVISÃO FINAL, APRESENTAÇÃO E CONFORMIDADE ÉTICA	
.....	14
6.1 Roteiro Estruturado para Apresentação em Vídeo (YouTube)	14
6.2 Alinhamento Estrito com o Código de Ética da Instituição EaD	15
REFERÊNCIAS	16

1 SEÇÃO 1: PLANEJAMENTO DA IMPLANTAÇÃO (PORTFOLIOHUB + IA GEMINI)

1.1 Definição do Escopo e Objetivos Gerais

A implantação do PortfolioHUB representa o estabelecimento de uma infraestrutura centralizada e unificada para o gerenciamento, visualização e auditoria de projetos e portfólios digitais. Este plano de implantação foi estruturado sob a égide da metodologia DevSecOps, integrando versionamento de código, automação de processos, segurança cibernética ativa e ferramentas de produtividade corporativa em nuvem.

O escopo técnico do projeto abrange a criação de uma interface de usuário responsiva e performática que consome dados operacionais em tempo real diretamente de ecossistemas externos, eliminando a dependência de bancos de dados relacionais pesados ou de infraestruturas de backend dispendiosas. Os objetivos estratégicos de longo prazo incluem:

- **Automação de Portfólio:**Garantir a automação completa no inventário e exibição de repositórios através do acoplamento dinâmico à API do GitHub.
- **Cultura de Segurança:**Implementar mecanismos robustos de controle de acesso de usuários e auditoria de código-fonte desde a primeira linha de código (Security by Design).
- **Integração Corporativa:**Integrar o canal de entrada de leads e feedback com as ferramentas de planilha e gestão do Google Workspace de maneira totalmente serverless.
- **Aceleração Cognitiva:**Utilizar o ecossistema de inteligência artificial generativa do Google Gemini como co-piloto e validador arquitetural de segurança.

1.2 Cronograma de Execução Crítica (Sprint de 48 Horas)

Dada a restrição temporal com prazo final estipulado para 14 de junho de 2026 às 23h55min, foi adotada uma abordagem de gerenciamento de projetos ágil (Scrum/Lean) em formato de Sprint de Alta Intensidade. O cronograma abaixo reflete a distribuição das macroatividades por blocos operacionais:

Período Cronológico	Fase Operacional	Atividades e Entregáveis Estratégicos
12/06/2026 Tarde e Noite	Fase 1: Setup, Infraestrutura e Escopo	Elaboração do plano descritivo detalhado; definição das stacks tecnológicas; ativação do prompt mestre no Gemini; provisionamento e inicialização do repositório remoto no GitHub contendo governança básica.
13/06/2026 Manhã e Tarde	Fase 2: Arquitetura e Integrações Core	Codificação estrutural da interface (HTML5/CSS3/JS); desenvolvimento do script assíncrono para consumo da API REST do GitHub; acoplamento do formulário de contato ao ecossistema do Google Workspace (Sheets/Forms).
13/06/2026 Noite	Fase 3: Políticas de DevSecOps e Controle	Configuração de regras estritas de proteção de branch (Branch Protection Rules) na main; ativação do Secret Scanning e Dependabot; simulação e atribuição de permissões baseadas em funções (RBAC).
14/06/2026 Manhã e Tarde	Fase 4: Garantia de Qualidade e Homologação	Execução da matriz completa de testes integrados (segurança de dados, sanitização de XSS, responsividade mobile e validação de persistência no Google Workspace); refinamento estético do código.
14/06/2026 Noite (Até 23h55)	Fase 5: Go-Live, Apresentação e Submissão	Fusão do código estável na branch principal; automação do deploy via GitHub Pages; gravação e publicação da apresentação técnica no YouTube; exportação e upload do relatório descritivo final em formato PDF.

1.3 Configuração e Engenharia de Prompt para o Google Gemini

A fim de maximizar a eficiência operacional e mitigar erros arquiteturais, a inteligência artificial generativa Google Gemini foi configurada como o co-piloto técnico oficial do projeto. O modelo atua sob uma diretriz de Engenharia de Prompt focada em contextos industriais avançados, eliminando alucinações de código e reforçando políticas de segurança da informação.

O 'Prompt Mestre de Ativação' foi estruturado para delimitar a persona do modelo, injetar as restrições normativas do desafio e parametrizar o formato das respostas técnicas recebidas durante toda a execução da trilha:

CONTEXTO E DIRETRIZ MASTER: Atue como um Arquiteto de Cloud, Especialista em Segurança da Informação e Engenheiro DevOps Sênior. Meu objetivo é projetar, documentar e implantar o PortfolioHUB integrado ao GitHub e ao ecossistema do Google Workspace, seguindo estritamente os critérios de avaliação da minha entrega final. Você deve me guiar passo a passo em todas as configurações práticas, fornecendo as melhores práticas de segurança de código (OWASP), comandos Git precisos sob a convenção de commits semânticos e checklists rigorosos de validação para cada etapa. Sempre priorize soluções robustas de frontend serverless, arquitetura limpa, documentação técnica transparente e total conformidade com códigos de ética acadêmica.

2 SEÇÃO 2: CONFIGURAÇÃO INICIAL E INTEGRAÇÃO COM GITHUB

2.1 Provisionamento do Ambiente Dedicado e Governança Inicial

A fase de execução prática inicia-se com o provisionamento de um repositório remoto público e dedicado na plataforma do GitHub, denominado formalmente como 'portfolio-hub'. A escolha da visibilidade pública justifica-se pela necessidade operacional de acoplamento do serviço gratuito de hospedagem estática GitHub Pages.

A governança inicial do código-fonte foi estabelecida na raiz do repositório através da criação de três artefatos indispensáveis: o arquivo 'README.md' para contextualização imediata do projeto, o arquivo '.gitignore' configurado para barrar o versionamento de arquivos locais redundantes ou variáveis locais sensíveis, e o arquivo contendo a licença open-source MIT, resguardando os direitos de autoria e transparência da entrega.

2.2 Estrutura Arquitetural de Pastas do Workspace

Para assegurar a manutenibilidade, escalabilidade e modularidade do código-fonte do PortfolioHUB, a estrutura de arquivos e diretórios foi estruturada de forma padronizada, dividindo claramente as responsabilidades de estilo, lógica de consumo e automação:

```
portfolio-hub/
├──
├── Actions)
│   ├── cd-deploy.yml # Automação de entrega contínua (GitHub
│   ├── style.css # Estilização visual (Tailwind CSS/Custom)
│   ├── github-api.js # Lógica assíncrona de consumo da API REST
│   └── images/ # Vetores, logotipos e mídias estáticas
├── index.html # Painel principal unificado (Dashboard)
├── README.md # Documentação técnica de arquitetura
├── CONTRIBUTING.md # Manual de conduta e diretrizes de
│   colaboração
└── .github/
    └── workflows/
        └── assets/
            ├── css/
            └── js/
```


2.3 Mecanismo de Integração Dinâmica via API REST do GitHub

O núcleo do PortfolioHUB baseia-se na eliminação de processos manuais de atualização de projetos. Em vez de registrar estaticamente cada repositório no arquivo HTML, a aplicação foi projetada para atuar como um espelho dinâmico do perfil do desenvolvedor, consumindo os endpoints da API REST pública do GitHub (v3).

O arquivo JavaScript 'github-api.js' executa uma requisição assíncrona baseada na `Fetch` API para o endereço 'https://api.github.com/users/{usuario}/repos?sort=updated&per_page=100'. O payload JSON retornado passa por um algoritmo de filtragem desenvolvido com auxílio do Gemini, cujos critérios são:

- **Exclusão de Forks:**Garantir que repositórios derivados de bifurcações de terceiros não poluam o painel principal.
- **Filtro por Tópicos:**Exibir apenas os projetos que contenham explicitamente a tag (topic) 'portfolio' definida no GitHub, funcionando como uma curadoria automatizada.
- **Extração de Metadados:**Capturar dados textuais descritivos, contagem de estrelas (engajamento) e a linguagem de programação predominante para exibição em tempo real.

2.4 Pipeline de Implantação e Deploy Contínuo (CI/CD)

O lançamento do PortfolioHUB foi estruturado utilizando o mecanismo do GitHub Pages, eliminando superfícies de ataque comuns de servidores (como SSH exposto ou painéis administrativos desprotegidos). O processo de deploy é orquestrado de forma totalmente automatizada por meio de um pipeline de Integração e Entrega Contínua (CI/CD) via GitHub Actions.

Configurado através do arquivo 'cd-deploy.yml', o fluxo automatizado monitora a branch principal. Toda vez que um Pull Request é aprovado e mesclado na branch 'main', o GitHub Actions inicializa um container virtual isolado, executa rotinas de

checagem sintática, compila os arquivos estáticos e publica a nova versão do PortfolioHUB em ambiente de produção em menos de 60 segundos.

3 SEÇÃO 3: GESTÃO DE USUÁRIOS E SEGURANÇA (DEVSECOPS)

3.1 Controle de Acesso Baseado em Regras (RBAC) e Fatores de Autenticação

Alinhado com o critério de avaliação de Robustez das Políticas de Segurança Implementadas, o PortfolioHUB incorpora o modelo RBAC (Role-Based Access Control) dentro de seu planejamento operacional. O acesso administrativo ao repositório centralizado segue rigorosamente o Princípio do Privilégio Mínimo (PoLP), mapeando os níveis de acesso de maneira estrita.

O autor original do projeto retém a credencial de 'Owner' (Administrador). Eventuais revisores, avaliadores ou membros de equipes de infraestrutura serão adicionados ao ecossistema com permissões severamente restritas sob a alcunha de 'Triage' ou 'Read', impossibilitando alterações não auditadas na árvore de produção. Adicionalmente, é estabelecida a obrigatoriedade da Autenticação de Múltiplos Fatores (MFA/2FA) via aplicativos leitores de OTP (One-Time Password) para mitigar o risco de sequestro de credenciais administrativas.

3.2 Políticas Robustas de Proteção de Código e Ramificações

Para blindar o ambiente produtivo contra inserções incorretas de código ou sabotagens internas, foram projetadas e ativadas as Regras de Proteção de Branch (Branch Protection Rules) aplicadas diretamente à ramificação 'main'. Estas regras de governança de código impedem comportamentos inadequados de desenvolvimento e são compostas pelos seguintes requisitos técnicos:

- **Proibição de Push Direto:** Bloqueio absoluto de operações de push direto de ambientes locais para a main.
- **Exigência de Pull Request:** Toda e qualquer alteração no código deve ser submetida obrigatoriamente através de uma branch de trabalho secundária e passar pelo crivo de um Pull Request.
- **Proibição de Force Push:** Bloqueio de deleções de histórico e force pushes, assegurando a integridade e a auditoria histórica temporal do projeto.

3.3 Varredura Automatizada de Vulnerabilidades e Vazamento de Segredos

A segurança do repositório é monitorada 24 horas por dia por meio da integração de ferramentas nativas de inteligência em segurança cibernética do GitHub Advanced Security, parametrizadas sob a consultoria da IA Gemini:

- **GitHub Secret Scanning:** Algoritmo de monitoramento contínuo que escaneia cada commit em milissegundos. Caso o desenvolvedor inclua acidentalmente uma chave privada de API, token de acesso pessoal ou credenciais em geral, o commit é bloqueado ou um alerta de severidade máxima é disparado instantaneamente, impedindo a exposição pública do segredo.
- **GitHub Dependabot Alerts:** Agente automatizado que inspeciona o grafo de dependências do projeto. O Dependabot cruza as bibliotecas declaradas no ecossistema com bancos de dados globais de vulnerabilidades conhecidas (CVEs). Caso uma vulnerabilidade seja detectada no frontend, a ferramenta abre de forma autônoma um Pull Request contendo a correção e a atualização de segurança.

3.4 Segurança de Frontend e Mitigação de Vetores de Ataque Web

Na camada de aplicação exposta ao usuário final no navegador, o PortfolioHUB adota blindagens contra o ataque Cross-Site Scripting (XSS), uma vulnerabilidade comum em aplicações que consomem dados de APIs de terceiros. Uma vez que o JavaScript buscará descrições e nomes de projetos vindos do GitHub (os quais poderiam ser manipulados por atores maliciosos), o código de renderização dinâmica foi explicitamente instruído pela IA Gemini a repudiar o uso da propriedade perigosa 'innerHTML'.

Toda a injeção de dados textuais no Document Object Model (DOM) é processada via propriedades seguras, como 'textContent' ou através da instanciação explícita de nós com 'document.createElement()'. Esta abordagem garante que qualquer tag de script maliciosa embutida nos dados retornados seja tratada puramente como texto inofensivo, neutralizando o ataque. Por fim, o canal de transmissão é inteiramente criptografado utilizando o protocolo TLS 1.3 / HTTPS forçado nativamente.

4 SEÇÃO 4: COMPARTILHAMENTO E CONTROLE DE ACESSO COM GITHUB

4.1 Fluxo de Trabalho de Engenharia de Código (Git Flow Simplificado)

O compartilhamento público do código-fonte e o gerenciamento de versões do PortfolioHUB obedecem a uma variação corporativa do Git Flow, otimizada para o modelo de desenvolvimento ágil contínuo. Este fluxo organiza o ciclo de vida do software em ramificações com papéis estritamente delineados:

- **Branch main (Produção):** Representa o ambiente de produção consolidado e imutável por vias normais. Contém exclusivamente código homologado, estável e pronto para consumo público.
- **Branches feature/* (Desenvolvimento):** Ramificações temporárias abertas exclusivamente para o desenvolvimento isolado de novas funcionalidades, correções de bugs ou melhorias de estilo (ex: feature/api-integration, feature/google-sheets-form). Uma vez concluído o escopo da funcionalidade, a branch é empurrada ao servidor remoto para abertura de inspeção visual (Pull Request) e posterior fusão com a main.

4.2 Padronização de Histórico via Commits Semânticos

Para viabilizar a rastreabilidade e permitir que avaliadores acadêmicos ou auditores de código compreendam a evolução do software em um único vislumbre, o projeto adota rigorosamente a convenção internacional de Conventional Commits (Commits Semânticos). Cada mensagem de envio de código deve possuir um prefixo estruturado indicando a natureza exata da alteração:

- **feat:** Para a introdução de uma nova funcionalidade no ecossistema (ex: 'feat: adiciona bloco de renderizacao da api do github').
- **fix:** Para a resolução de uma falha, bug ou comportamento incorreto do sistema (ex: 'fix: corrige estouro de layout nos cards para telas mobile').
- **docs:** Para modificações exclusivas em arquivos de documentação técnica (ex: 'docs: atualiza guia de instalacao no readme').

- **style:** Para alterações de formatação visual, espaçamentos ou design que não alteram o comportamento lógico do sistema (ex: 'style: aplica classes do tailwind para botoes').
- **refactor:** Para otimizações e melhorias na estrutura interna do código sem alterar suas funções externas (ex: 'refactor: otimiza funcao assincrona de fetch').

4.3 Artefatos de Governança Comunitária e Normas de Colaboração

O compartilhamento seguro e o controle de acesso transparente exigem uma documentação que declare as regras do jogo para a comunidade externa. O PortfolioHUB materializa essa clareza através de dois arquivos fundamentais hospedados na raiz do repositório:

O 'README.md' atua como o manual técnico do produto, detalhando a arquitetura serverless, a stack de tecnologias, pré-requisitos locais, variáveis de ambiente configuradas e os links hiperlinkados da aplicação em produção. Já o arquivo 'CONTRIBUTING.md' explicita o guia comportamental e técnico para novos colaboradores, instruindo sobre a obrigatoriedade do uso de commits semânticos, abertura correta de Pull Requests e cumprimento do código de conduta do projeto.

5 SEÇÃO 5: FINALIZAÇÃO DA INTEGRAÇÃO E TESTES

5.1 Matriz de Testes Integrados e Homologação de Funcionalidades

A fase final de integração consiste na submissão da plataforma a um plano abrangente de garantia de qualidade (Quality Assurance) estruturado com apoio analítico do Google Gemini. A homologação do PortfolioHUB é condicionada à aprovação em três baterias de testes rigorosos:

- **Teste de Integração da API REST:** Validar se as chamadas assíncronas capturam com precisão os dados do GitHub, testar a resiliência do sistema perante cenários de instabilidade na API (exibição de mensagens de fallback adequadas) e aferir o comportamento frente às restrições de taxa de requisição (Rate Limiting).
- **Teste de Persistência no Google Workspace:** Submeter dados de teste no formulário de contato frontend e certificar que a cadeia de caracteres é transmitida de forma íntegra e serverless para as planilhas do Google Sheets, validando a automação de leads corporativos no Workspace.
- **Teste de Interface, Layout e Acessibilidade:** Inspecionar visualmente o site em múltiplas resoluções de tela utilizando ferramentas de emulação de dispositivos móveis, garantindo fluidez e responsividade, além de executar varreduras automatizadas para detecção de links quebrados.

5.2 Validação de Segurança Pré-Lançamento (Audit Go-Live)

Imediatamente antes do congelamento de código para distribuição pública, é realizada uma auditoria de segurança pré-lançamento de 'Portão de Qualidade Final' (Quality Gate). Esta rotina visa atestar que as diretrizes planejadas na Seção 3 foram cumpridas integralmente.

Com auxílio do Gemini, executa-se uma varredura manual e automatizada no histórico de commits para garantir que nenhum Token de Acesso Pessoal (PAT) do GitHub ou link privado com privilégio de escrita do Google Workspace tenha sido deixado nos códigos JavaScript expostos ao cliente. Adicionalmente, valida-se o correto

redirecionamento de tráfego HTTP para HTTPS e a ausência de vulnerabilidades de injeção direta no DOM.

5.3 Transição de Ambientes e Acessibilidade Pública em Produção

Após a validação em sede de testes locais, o código-fonte estável é unificado na branch principal por meio da aprovação do Pull Request final. O gatilho automático de CI/CD é acionado e transfere os artefatos validados para o ambiente real de produção hospedado sob o domínio do GitHub Pages.

A plataforma passa a operar sob uma URL pública acessível globalmente (ex: <https://seu-usuario.github.io/portfolio-hub/>). Uma verificação final é conduzida através de acessos em navegadores em modo anônimo e redes externas isoladas, garantindo que o avaliador da banca acadêmica consiga acessar e navegar por todos os projetos e funcionalidades sem enfrentar barreiras de permissão ou falhas de carregamento.

6 SEÇÃO 6: REVISÃO FINAL, APRESENTAÇÃO E CONFORMIDADE ÉTICA

6.1 Roteiro Estruturado para Apresentação em Vídeo (YouTube)

Cumprindo a última etapa regulamentar do desafio, foi elaborado um roteiro de gravação técnico e dinâmico para a apresentação em vídeo a ser disponibilizada publicamente no YouTube. O roteiro foi planejado para cobrir com precisão metodológica os pontos altos da implantação dentro de uma janela temporal enxuta:

- **Bloco 1 - Introdução e Visão Geral:** Introdução pessoal; contextualização do desafio; apresentação do PortfolioHUB e justificativa da arquitetura serverless adotada (Tempo estimado: 01m30s).
- **Bloco 2 - Integração e Engenharia Prática:** Demonstração prática do site em funcionamento; exibição dos cards de projetos sendo renderizados dinamicamente a partir da API REST do GitHub; preenchimento do formulário de contato e exibição instantânea dos dados sendo armazenados no Google Sheets do Workspace (Tempo estimado: 03m00s).
- **Bloco 3 - Segurança da Informação (DevSecOps):** Explicação detalhada das políticas de segurança implementadas; demonstração das Branch Protection Rules no GitHub; menção à proteção XSS no frontend e às varreduras do Dependabot e Secret Scanning (Tempo estimado: 02m30s).
- **Bloco 4 - Desafios Superados e Conclusão:** Discussão sobre as principais barreiras técnicas encontradas (ex: tratamento de limites da API ou design responsivo); soluções encontradas com suporte analítico da IA Gemini; considerações finais e encerramento (Tempo estimado: 02m00s).

6.2 Alinhamento Estrito com o Código de Ética da Instituição EaD

O desenvolvimento, planejamento e execução prática do PortfolioHUB foram inteiramente pautados pelas diretrizes de integridade acadêmica e conduta profissional estipuladas no Código de Ética da instituição de Ensino a Distância (EaD). Embora o ecossistema Google Gemini tenha sido utilizado de forma intensiva, sua aplicação deu-

se exclusivamente sob o papel de co-piloto de engenharia, acelerador cognitivo e validador arquitetural.

Toda a lógica de negócios, design estrutural, organização de pastas e redação técnica contida neste documento foram concebidas e executadas pelo estudante graduando, configurando autoria própria legítima. Elementos de código de terceiros ou bibliotecas de código aberto utilizadas no frontend foram devidamente creditados e referenciados no repositório, repudiando veementemente práticas de plágio, cópia não autorizada ou fraude acadêmica, assegurando uma entrega em total conformidade com a ética universitária.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: Informação e documentação - Trabalhos acadêmicos - Apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: Informação e documentação - Referências - Elaboração. Rio de Janeiro: ABNT, 2018.

GITHUB. GitHub REST API Documentation. Disponível em: <<https://docs.github.com/en/rest>>. Acesso em: 12 jun. 2026.

GITHUB. About branch protection rules. Disponível em: <<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-branch-protection-rules>>. Acesso em: 12 jun. 2026.

GOOGLE. Google Workspace Apps Script API Reference. Disponível em: <<https://developers.google.com/apps-script>>. Acesso em: 12 jun. 2026.

OWASP FOUNDATION. OWASP Top 10:2021 - The Most Critical Web Application Security Risks. Disponível em: <<https://owasp.org/www-project-top-ten/>>. Acesso em: 12 jun. 2026.